Graphes & Applications

Résumé de cours — Chapitres 0 à 4

Table des matières

1 Introduction à la modélisation par les graphes **1**
1.1 Contexte : origine et recherche opérationnelle . . . 1
1.2 Graphes non orientés . . . 2
1.3 Graphes orientés . . . 2
1.4 Graphes valués et représentations . . . 3

2 Coloration des sommets d'un graphe **3**
2.1 Modélisation . . . 3
2.2 Propriétés et encadrement de $\chi(G)$ . . . 3
2.3 Algorithme glouton . . . 4
2.4 Algorithme de Welsh & Powell . . . 4

3 Recherche de plus court chemin **4**
3.1 Définitions . . . 4
3.2 Algorithme de Dijkstra (poids positifs) . . . 5
3.3 Algorithme de Ford–Bellman (poids quelconques) . . . 5
3.4 Algorithme de Floyd–Warshall (tous les couples) . . . 5
3.5 Comparaison des algorithmes . . . 6

4 Flots sur les réseaux **6**
4.1 Réseau de transport . . . 6
4.2 Algorithme de Ford–Fulkerson . . . 7
4.3 Coupe minimale . . . 7

5 Arbre couvrant à poids minimal **7**
5.1 Définitions et propriétés . . . 8
5.2 Algorithme de Kruskal . . . 8
5.3 Algorithme de Prim . . . 8

6 Aide-mémoire **9**

1 Introduction à la modélisation par les graphes

1.1 Contexte : origine et recherche opérationnelle

— **Origine** : en 1735, Euler résout le problème des sept ponts de Königsberg (traverser chaque pont une seule fois) et prouve que c'est impossible : un tel parcours exige que chaque sommet ait un degré pair (sauf deux au maximum, pour un parcours qui ne finit pas où il commence).

À Königsberg, tous les sommets avaient un degré impair. C'est la naissance de la théorie des graphes.

— **Recherche opérationnelle (RO)** : outil d'aide à la décision ; ensemble de méthodes mathématiques et algorithmiques pour résoudre un problème de la meilleure façon possible (optimiser, réduire les coûts). Née pendant la seconde guerre mondiale (logistique militaire). Applications : production, transport, télécommunications, planification de projets, gestion financière, allocation de fréquences GSM ou de niveaux de vol.

1.2 Graphes non orientés

Définition (Graphe non orienté)

$G = (V, E)$ avec $V = \{v_1, \ldots, v_n\}$ l'ensemble des **sommets** ($|V| = n$, *ordre* du graphe) et $E = \{e_1, \ldots, e_p\}$ l'ensemble des **arêtes** ($|E| = p$).

Notion	Définition
Adjacents / voisins	Deux sommets reliés par une arête
Sommet isolé	Sommet adjacent à aucun autre
Arête incidente	e est incidente à v si v est une extrémité de e ; deux arêtes incidentes à un même sommet sont adjacentes
Boucle	Arête dont les deux extrémités sont égales
Arêtes multiples (parallèles)	Plusieurs arêtes reliant les mêmes sommets
Sous-graphe	$G' = (V', E')$ avec $V' \subseteq V$ et $E' \subseteq E$
Graphe partiel	$G' = (V', E')$ avec $V' = V$ et $E' \subseteq E$
Graphe simple	Au plus une arête entre deux sommets, pas de boucle (*seul type utilisé dans le cours*)
Multigraphe / pseudo-graphe	Arêtes multiples autorisées / boucles et arêtes multiples autorisées
Chaîne / cycle	Suite de sommets reliés par des arêtes / chaîne fermée dont les arêtes sont distinctes
Graphe connexe	Pour tous x, y, il existe une chaîne de x vers y ; sinon, il est réunion de **composantes connexes** (sous-graphes connexes maximaux)
Graphe complet K_n	Tous les sommets sont reliés deux à deux ; il a $\frac{n(n-1)}{2}$ arêtes
Graphe biparti	Sommets partitionnés en V_1, V_2 avec toutes les arêtes entre V_1 et V_2
Graphe régulier	Tous les sommets ont le même degré

À retenir : degré

Le **degré** $d(v_i)$ est le nombre d'arêtes incidentes à v_i (une boucle compte double ; dans un graphe simple, c'est le nombre de voisins). Le **degré du graphe** est le degré maximum de ses sommets. **Lemme des poignées de mains** : $\sum_{v \in V} d(v) = 2|E|$.

1.3 Graphes orientés

— Un graphe orienté $G = (V, E)$ a des **arcs** $e = (u, v)$ (paire ordonnée) : u extrémité initiale, v extrémité finale ; v est un *successeur* de u et u un *prédécesseur* de v.

— **Degré extérieur** $d^+(v)$: nombre d'arcs d'extrémité initiale v ; **degré intérieur** $d^-(v)$: nombre d'arcs d'extrémité finale v ; $d(v) = d^+(v) + d^-(v)$.

— **Chemin** (analogue de la chaîne) ; **circuit** : chemin fermé (analogue du cycle).
— **Fortement connexe** : pour tout couple (u, v), il existe un chemin de u à v *et* de v à u. **Faiblement connexe** : connexe si l'on ignore l'orientation.

1.4 Graphes valués et représentations

Un **graphe valué** associe à chaque arête ou arc (u, v) un poids $w(u, v)$ (coût, distance, capacité…).

Représentation	Description
Matrice d'adjacence	Matrice $n \times n$: $A[i][j] = 1$ (ou le poids) si l'arête $v_i v_j$ existe, 0 sinon ; *symétrique* pour un graphe non orienté
Matrice d'incidence sommets-arcs	Lignes = sommets, colonnes = arcs, coefficients dans $\{0, +1, -1\}$: pour l'arc $u = (i, j)$, $a_{iu} = +1$, $a_{ju} = -1$, autres termes nuls
Listes d'adjacence	Table où chaque ligne (un sommet) contient la liste de ses successeurs (ou prédécesseurs) ; efficace quand le nombre d'arêtes est faible

Critères d'efficacité d'un algorithme : le *temps* de calcul et l'*espace* mémoire utilisé.

2 Coloration des sommets d'un graphe

2.1 Modélisation

Exemple : attribution des quais d'un port

Deux navires dont les horaires se chevauchent ne peuvent pas partager un quai. On modélise : **sommet** = navire, **arête** = chevauchement des horaires, **couleur** = quai. Le nombre minimal de couleurs est le nombre minimal de quais.

Définition (Coloration, nombre chromatique)

Colorier un graphe, c'est colorier ses sommets de sorte que deux sommets adjacents aient des couleurs différentes. Le **nombre chromatique** $\chi(G)$ est le plus petit nombre de couleurs nécessaires.

2.2 Propriétés et encadrement de $\chi(G)$

À retenir : encadrer le nombre chromatique

Pour $G = (V, E)$ avec $|V| = n$:
— Graphe complet : $\chi(K_n) = n$.
— **Minoration** : si G contient un sous-graphe complet K_p, alors $p \leq \chi(G)$.
— **Majoration** : $\chi(G) \leq n$; si l'on exhibe une coloration valide à p couleurs, $\chi(G) \leq p$; **Brooks** : si d est le plus grand degré, $\chi(G) \leq d + 1$.
— **Graphe planaire** (dessinable dans le plan sans croisement d'arêtes, ex. une carte : sommet = région, arête = frontière) : $\chi(G) \leq 4$ (théorème des quatre couleurs, Appel–Haken, 1977).
Si minorant et majorant coïncident, on a $\chi(G)$; sinon on ne peut pas conclure.

2.3 Algorithme glouton

Algorithme : Glouton

Principe : avancer étape par étape en faisant un choix localement optimal, sans souci d'optimalité globale.

1. Parcourir les sommets dans un ordre donné.
2. À chaque sommet, affecter la **première couleur possible**, c'est-à-dire la plus petite couleur non utilisée par ses voisins déjà colorés.

Il nécessite n étapes et utilise au plus $d+1$ couleurs (d : degré du graphe).

2.4 Algorithme de Welsh & Powell

Algorithme : Welsh & Powell

1. **Ordonner** les sommets par degré décroissant : $\deg(v_1) \geq \deg(v_2) \geq \cdots \geq \deg(v_n)$.
2. Affecter la couleur C_1 à v_1, puis la même couleur à chaque sommet suivant de la liste **non colorié et non adjacent** à un sommet déjà de couleur C_1.
3. S'il reste des sommets non coloriés, prendre une **nouvelle couleur** pour le premier sommet non colorié de la liste résiduelle, et recommencer.

Arrêt : tous les sommets sont coloriés.

Exemple : tableau de Welsh & Powell

Sommets dans l'ordre A, B, E, F, C, D (degrés 4, 4, 4, 4, 3, 3). Itération 1 : C_1 pour A et C ; itération 2 : C_2 pour B et D ; itération 3 : C_3 pour E ; itération 4 : C_4 pour F. Donc $\chi(G) \leq 4$.

Remarque

Ni Glouton ni Welsh & Powell ne sont toujours optimaux : changer l'ordre de visite de deux sommets peut faire passer le glouton de 4 à 3 couleurs (avec un K_3 présent, on obtient alors $\chi(G) = 3$) ; des contre-exemples existent aussi pour Welsh & Powell (une coloration à 2 couleurs existe alors que l'algorithme en donne davantage). Ces algorithmes fournissent donc un *majorant* de $\chi(G)$.

3 Recherche de plus court chemin

Exemples : itinéraires (Google Maps), routage de paquets dans un réseau.

3.1 Définitions

Définition (Graphe pondéré, longueur d'un chemin)

Un graphe pondéré est muni d'une application $p : E \to \mathbb{R}$ (poids). La **longueur** d'un chemin $m = \langle v_1, \ldots, v_k \rangle$ est la somme des poids de ses arcs : $l(m) = \sum_{i=1}^{k-1} p(v_i, v_{i+1})$.

Définition (Distance, circuit absorbant)

La **distance** $\delta(v_i, v_j)$ est la plus petite longueur des chemins de v_i à v_j ; $\delta = +\infty$ s'il n'existe aucun chemin ; $\delta = -\infty$ s'il existe un chemin contenant un circuit de coût total négatif. Un **circuit absorbant** est un circuit de coût total négatif.

À retenir

Condition nécessaire d'existence d'un plus court chemin : **absence de circuit absorbant**. *Sous-chemins optimaux* : tout sous-chemin d'un plus court chemin est un plus court chemin.

Le choix de l'algorithme dépend des poids (positifs ou de signe quelconque) et du problème : un sommet vers un autre, un sommet vers tous les autres, ou tous les couples de sommets.

3.2 Algorithme de Dijkstra (poids positifs)

Plus courts chemins d'une source x_1 vers tous les sommets. On note $\text{dist}(j)$ la longueur optimale de x_1 à x_j, P l'ensemble des sommets à distance *permanente* (plus court chemin déterminé), $T = V \setminus P$ les sommets à distance *temporaire* ; à chaque itération, un sommet passe de T à P.

Algorithme : Dijkstra–Moore, 1959

1. **Initialisation** : $\text{dist}(1) = 0$; pour $j \geq 2$, $\text{dist}(j) = C_{1,j}$ si $(x_1, x_j) \in A$, $+\infty$ sinon ; $\text{Pcc}(1) = (x_1)$, $P = \{x_1\}$, $T = \{x_2, \ldots, x_n\}$.
2. **Tant que** $T \neq \emptyset$:
   - *Choix* : prendre $x_k \in T$ avec $\text{dist}(k) = \min_{x_j \in T} \text{dist}(j)$; mémoriser $\text{Pcc}(k)$, $P \leftarrow P \cup \{x_k\}$, $T \leftarrow T \setminus \{x_k\}$.
   - *Mise à jour* : pour chaque successeur $x_j \in T$ de x_k, si $\text{dist}(k) + C_{k,j} < \text{dist}(j)$, alors $\text{dist}(j) \leftarrow \text{dist}(k) + C_{k,j}$ (et le chemin est mis à jour).
3. Afficher dist et Pcc (on obtient une *arborescence* optimale).

À retenir : limite de Dijkstra

Dijkstra **n'est plus valable avec des poids négatifs** : il fixe définitivement une distance trop tôt. Sur l'exemple du cours, il renvoie $\text{dist}(3) = 8$ alors que le chemin $(1, 2, 5, 3)$ donne $\text{dist}(3) = 5$. D'où l'algorithme de Ford–Bellman.

3.3 Algorithme de Ford–Bellman (poids quelconques)

Graphe à coûts quelconques ; plus court chemin d'un sommet vers tous les autres ; converge en l'absence de circuit absorbant. $\text{dist}^k(j)$ désigne la longueur optimale d'un chemin de x_1 à x_j comportant au plus k sommets autres que l'origine.

Algorithme : Ford–Bellman

1. **Étape 0** : $k = 1$; $\text{dist}^k(1) = 0$; pour $j \geq 2$, $\text{dist}^1(j) = C_{1,j}$ si $(x_1, x_j) \in A$, $+\infty$ sinon.
2. **Étape 1 (mise à jour)** : pour $j = 1, \ldots, n$, $\text{dist}^{k+1}(j) = \min\Big(\text{dist}^k(j),\ \min_l \big(\text{dist}^k(l) + C_{l,j}\big)\Big)$, où x_l parcourt les prédécesseurs de x_j.
3. **Étape 2 (test de convergence)** : si $\text{dist}^{k+1}(j) = \text{dist}^k(j)$ pour tout j, *fin* (optimalité atteinte) ; sinon, si $k = n$, il existe un **circuit absorbant** ; sinon $k \leftarrow k + 1$ et retour à l'étape 1.

3.4 Algorithme de Floyd–Warshall (tous les couples)

Généralisation de Ford–Bellman : plus court chemin entre *tous* les couples de sommets, à partir de la matrice des poids ($+\infty$ si l'arc n'existe pas).

Algorithme : Floyd–Warshall

1. **Initialisation** : $\text{dist}(i,j) = 0$ si $i = j$, $C_{i,j}$ si $(x_i, x_j) \in A$, $+\infty$ sinon ; $\text{Pcc}(i,j) = (x_i, x_j)$.
2. **Pour** $k = 1, \ldots, n$ (insertion du sommet k) et pour tous $i \neq k$, $j \neq k$: si $\text{dist}(i,k) + \text{dist}(k,j) < \text{dist}(i,j)$, alors $\text{dist}(i,j) \leftarrow \text{dist}(i,k) + \text{dist}(k,j)$ et $\text{Pcc}(i,j) \leftarrow \text{Pcc}(i,k) \cup \text{Pcc}(k,j)$.
3. **Test** : s'il existe i avec $\text{dist}(i,i) < 0$, il existe un **circuit absorbant** ; sinon, optimalité atteinte à $k = n$.

3.5 Comparaison des algorithmes

	Dijkstra	**Ford–Bellman**	**Floyd–Warshall**
Chemins calculés	d'un sommet vers les autres	d'un sommet vers les autres	toutes les paires
Complexité, listes d'adjacence	$\Theta(n + m \log n)$	$\Theta(n \times m)$	—
Complexité, matrice d'adjacence	$\Theta(n^2)$	$\Theta(n^3)$	$\Theta(n^3)$
Cas d'utilisation	poids positifs	poids quelconques	poids quelconques

4 Flots sur les réseaux

4.1 Réseau de transport

Définition (Réseau de transport)

Graphe orienté sans circuit $G = (V, E)$ dont chaque arc e a une **capacité** $C_e \geq 0$, avec une **source** s (sommet sans prédécesseur, de degré intérieur nul) et un **puits** t (sommet sans successeur, de degré extérieur nul). Les autres sommets sont des nœuds de transit.

Définition (Flux, flot)

Le **flux** φ_e est la quantité circulant sur l'arc $e = (i,j)$; un **flot** est le vecteur des flux de tous les arcs.

À retenir : flot réalisable (admissible)

Un flot est réalisable s'il vérifie :
- la **contrainte de capacité** : $0 \leq \varphi_e \leq C_e$ pour tout arc e ;
- la **conservation du flux** en tout nœud de transit : $\sum_{i \in P(j)} \varphi(i,j) = \sum_{i \in S(j)} \varphi(j,i)$ (flux entrant = flux sortant).

Notion	**Définition**
Arc de retour	Arc fictif (t,s) de capacité infinie : son flux est la *valeur de retour* ; la conservation est alors valable pour tous les nœuds
Valeur du flot	Quantité totale qui traverse le réseau
Arc saturé	$\varphi_e = C_e$
Chemin saturé	Chemin passant par au moins un arc saturé
Flot complet	Tout chemin de s à t est saturé

Définition (Problème du flot maximal)

Trouver la plus grande quantité de flot réalisable que l'on peut acheminer de s à t en respectant les capacités. Applications : distribution d'eau, réseaux de communication (volume de données), réseaux de production (usine → entrepôt).

4.2 Algorithme de Ford–Fulkerson

Définition (Graphe résiduel)

Associé à un flot φ : pour un arc direct (x_i, x_j), **capacité résiduelle** $C_{i,j} - \varphi_{i,j}$; pour un arc inverse (créé fictivement), capacité égale au flux $\varphi_{i,j}$ (possibilité de *défaire* du flux).

Algorithme : Ford–Fulkerson

1. **Étape 0** : flot initial réalisable φ^0 (par exemple le flot nul).
2. **Étape 1 (flot complet)** : chercher, par exploration en profondeur, un *chemin non saturé* (augmentant) p^k de s à t dans le graphe résiduel. S'il existe : Δ_k = capacité résiduelle minimale sur p^k, $\varphi^{k+1} = \varphi^k + \Delta_k$, et mise à jour du graphe résiduel (flux résiduel $r_{k+1,a} = r_{k,a} - \Delta_k$ sur chaque arc a de p^k) ; $k \leftarrow k+1$. Sinon, passer à l'étape 2.
3. **Étape 2 (chaînes améliorantes)** : chercher une *chaîne améliorante* p^k de s à t dans le graphe résiduel (pouvant utiliser des arcs inverses). Si elle existe : $\varphi^{k+1} = \varphi^k + \Delta_k$, avec $r_{k+1,a} = r_{k,a} - \Delta_k$ si a est un arc direct et $r_{k+1,a} = r_{k,a} + \Delta_k$ si a est un arc inverse ; $k \leftarrow k+1$. Sinon, **fin : le flot est maximal**.

4.3 Coupe minimale

Définition (Coupe)

Une **coupe** $[S, T]$ est une partition $V = S \cup T$, $S \cap T = \emptyset$, avec $s \in S$ et $t \in T$. Sa **capacité** est la somme des capacités des arcs allant de S vers T : $\text{cap}[S,T] = \sum_{i \in S,\, j \in T} C_{ij}$.

À retenir : flot max et coupe min

Pour toute coupe : $\varphi_{\max} \leq \text{cap}[S,T]$. Si $\text{cap}[S,T] = \varphi_{\max}$, la coupe est **minimale**.
Construction : calculer d'abord le flot maximal, puis mettre dans S la source et tous les sommets atteignables depuis s par des chemins (chaînes) non saturés dans le graphe résiduel ; les autres sommets vont dans T. Les arcs de S vers T sont les **arcs bloquants**.

Exemple : distribution d'eau potable

Capacités (m^3/h) : $S \to A$ 30, $S \to B$ 20, $A \to C$ 20, $A \to D$ 15, $B \to C$ 10, $D \to T$ 25, $C \to T$ 20. Le puits reçoit au plus 15 par D (car D n'est alimenté que par $A \to D$ de capacité 15) et 20 par C : le flot maximal est **35** m^3/h (donc impossible d'envoyer 50). La coupe minimale est $S = \{S, A, B, C\}$, $T = \{D, T\}$, de capacité $15 + 20 = 35$. Les conduites bloquantes sont $A \to D$ et $C \to T$: ce sont les seules à agrandir pour augmenter le débit, inutile de remplacer tout le réseau (au-delà d'un certain agrandissement, une autre coupe deviendra limitante).

5 Arbre couvrant à poids minimal

Problème historique : en 1926, Otakar Borůvka conçoit un réseau de distribution électrique pour la Moravie du Sud au moindre coût. Les algorithmes de **Kruskal** (1956) et de **Prim** (1957) sont des

algorithmes **gloutons**. Applications : relier n villes par un réseau routier de coût minimal, relier des ordinateurs avec la plus petite longueur de câble.

5.1 Définitions et propriétés

On travaille avec des graphes **non orientés et valués**.

Définition (Arbre, arbre couvrant)

Un **arbre** est un graphe connexe sans cycle ; un arbre à n sommets a $n-1$ arêtes. Un **arbre couvrant** de $G=(V,E)$ est un graphe partiel $T=(V,E')$, $E'\subseteq E$, connexe et sans cycle. Le **poids** d'un arbre est la somme des poids de ses arêtes.

Définition (ACM)

Un **arbre couvrant de poids minimum** (ACM, *Minimum Spanning Tree*) est un arbre couvrant dont le poids est le plus petit possible. Tout graphe non orienté, valué et connexe admet un ou plusieurs ACM.

À retenir : propriétés équivalentes

Pour un graphe G à n sommets et m arêtes, sont équivalentes : G connexe sans cycle ; G connexe et $m=n-1$; il existe un unique chemin entre deux sommets quelconques. Un graphe non connexe n'a aucun arbre couvrant ; un graphe connexe en a au moins un.

5.2 Algorithme de Kruskal

Algorithme : Kruskal

Principe : ranger les arêtes par poids croissant et les ajouter une à une à l'ACM tant que leur ajout ne crée pas de cycle.

1. $T=\emptyset$; trier les arêtes de G par poids croissant.
2. Traiter les arêtes dans cet ordre : si l'arête **connecte deux composantes connexes** de T, l'ajouter à T ; sinon l'ignorer.
3. S'arrêter dès que T est connexe (on a incorporé $n-1$ arêtes) ; retourner T.

Exemple : Kruskal sur un graphe à 9 sommets $a,\dots,i$

Arêtes triées (poids) : a-g (1) oui, b-h (1) oui, f-i (1) oui, i-d (1) oui, g-b (2) oui, a-b (2) non, e-d (2) oui, b-c (3) oui, h-i (3) oui, puis i-e (3), h-f (3), i-c (4), g-f (5), c-d (5) refusées (cycles). ACM : a-g, b-h, f-i, i-d, g-b, e-d, b-c, h-i ; poids total $1+1+1+1+2+2+3+3=\mathbf{14}$.

5.3 Algorithme de Prim

Algorithme : Prim

Principe : partir d'un arbre réduit à un sommet, puis l'agrandir à chaque itération en le connectant à son plus proche voisin libre.

1. Initialiser T avec un sommet de G choisi et aucune arête.
2. **Répéter** : parmi les arêtes reliant un sommet de T à un sommet extérieur, choisir une arête de poids **minimal** ; ajouter à T cette arête et le sommet correspondant.
3. S'arrêter quand tous les sommets sont dans T ; retourner T.

Exemple : Prim sur le même graphe, départ en a

Choix successifs : a-g (1), a-b (2), b-h (1), b-c (3), h-f (3), f-i (1), i-d (1), d-e (2) ; les arêtes qui fermeraient un cycle sont écartées (g-b, g-f, c-i). Poids total $1+2+1+3+3+1+1+2 = \mathbf{14}$, le même que Kruskal (l'ACM n'est pas unique, mais son poids l'est).

	Kruskal	**Prim**
Idée	Trier les arêtes ; fusionner des composantes	Faire grossir un seul arbre depuis un sommet
Test à chaque pas	L'arête ne crée pas de cycle	L'arête relie T à l'extérieur, poids minimal
Arrêt	$n-1$ arêtes retenues	Tous les sommets dans T

6 Aide-mémoire

Thème	**À retenir**
Degrés	$\sum d(v) = 2\lvert E\rvert$; K_n a $n(n-1)/2$ arêtes ; orienté : $d = d^+ + d^-$
Coloration	$p \le \chi(G) \le d+1$ (sous-graphe K_p, Brooks) ; planaire : $\chi \le 4$; Glouton et Welsh & Powell donnent un *majorant*
Dijkstra	Poids ≥ 0 ; un sommet $\to$ tous ; $\Theta(n^2)$ (matrice)
Ford–Bellman	Poids quelconques ; détecte les circuits absorbants (si $k = n$) ; $\Theta(n\,m)$
Floyd–Warshall	Tous les couples ; $\text{dist}(i,i) < 0 \Rightarrow$ circuit absorbant ; $\Theta(n^3)$
Flots	Capacité + conservation ; graphe résiduel ; $\varphi_{\max} = \text{cap(coupe minimale)}$
ACM	Arbre : connexe, sans cycle, $n-1$ arêtes ; Kruskal (trier les arêtes) et Prim (faire grossir un arbre) : algorithmes gloutons